

1) Introduction

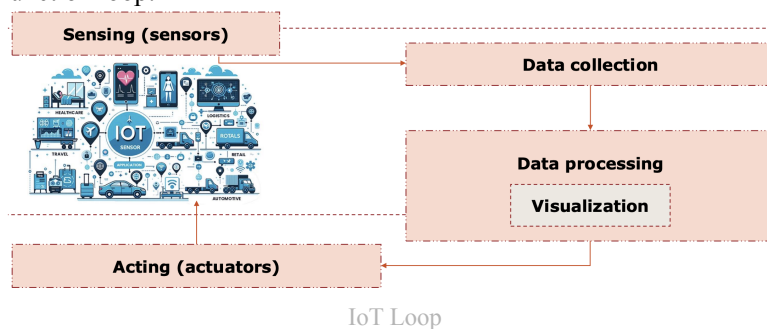
Definition 1 (Internet Of Things (IoT)).

Connecting everything

In particular we put emphasis on the internet (IP) side: no IP $\implies$ no IoT

"Internet of Things is a paradigm according to which every thing, real or virtual, is assigned an IP(v6) address and can be reached (for example for sensing or actuating purposes) via the standard Internet Protocol stack."

IoT has the following function loop:



Data Collection:

A sensor **monitors** and **reports states** of a physical entity (periodic, on demand, under certain conditions) and produces a **digital representation** suitable for the cyberspace. The first step in data acquisition is **metadata** to aid data annotation by describing nature/context of capture.

Processing and Visualization

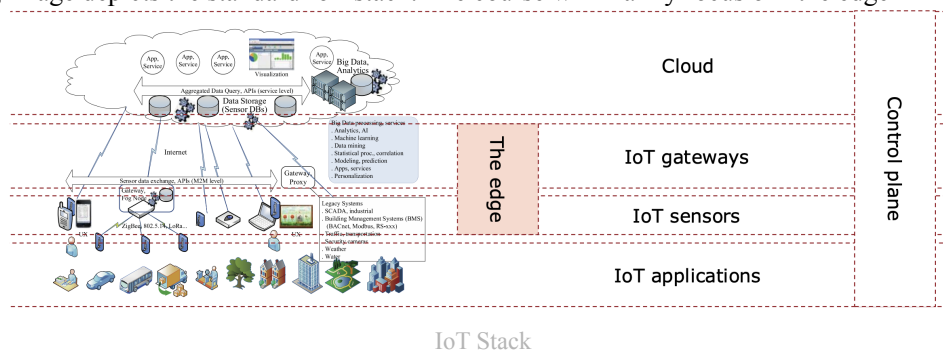
A **simple control loop** can be implemented and performs action directly on data as it arrives.

Then an **analysis** of data using algorithms/ai is performed. The data is processed in a series of steps. It can be finally displayed via a digital twin/dashboard

Acting

TODO

The following image depicts the standard IoT stack. The course will mainly focus on "the edge"



The whole stack is called control plane as it allows for the main IoT functionalities and infrastructure (service/security/performance management)

Applications

The applications are all the sectors in which IoT can be applied. In particular the biggest share is the Industrial IoT (IIoT).

The Edge

SENSORS

Sensors actually include both **sensors and actuators** where one measures and the other does something in the environment. Some sensors can do **local operations** while others need to **connect to the cloud** before communicating with actuators.

A sensor detects a measurement and converts it as a processable output. There are three types:

- **Fixed:** stable position wrt network topology. No re-authentication stable routing but must be correctly deployed
- **Mobile:** connects to cellular/LPWA network. This cannot be mains powered, and has limited mobility constraints
- **Nomadic:** can change position, but doesn't move while it communicates. Must re-authenticate
- **Active:** needs power source to work
- **Passive:** the measurement is done by the phenomenon itself it doesn't need power

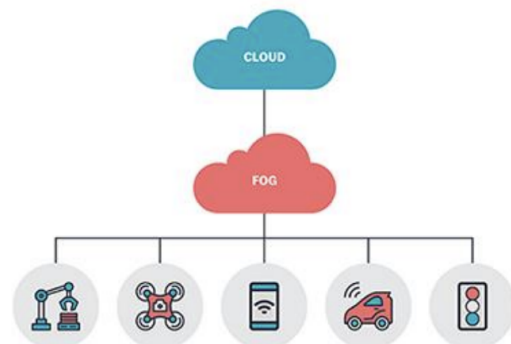
An actuator performs actions when/how dictated by the algorithm/operator.

- **Digital:** on-off action
- **Analog:** continuous signal to drive devices

GATEWAYS

Sensors connect to the internet using **gateways** which are usually more powerful devices. Links edge devices with cloud. It performs/assists with data collection, connectivity and security

A gateway can implement **fog nodes** by "bringing the cloud to the ground". That is, they allow for additional computation/storage capabilities closer to the edge. Gateways can store data and send only important samples to the cloud. They can also execute simple, low-latency commands (ITTT)



Fog Diagram

Cloud

The cloud is the **aggregated and processed** sources for optimization and discovery and analytics

1.2) Architecture

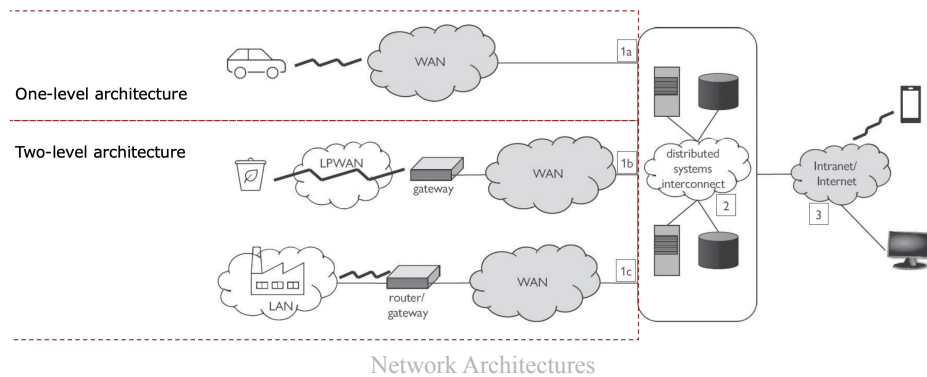
The architecture works on some main assumptions:

- **Client/Server** explicitly know the destination of the messages
- **Publish/Subscribe:** sources publish their messages in a logical channel in which one or more devices can subscribe to receive the message. This is a **one to many communication pattern**
 - There are 3 main flows: **async events** and **periodic/on-demand events** (uplink) and **commands**

(downlink)

- The main properties are: trustworthiness, architecture and functional

Architecture



The **one level architecture (1a)** consists in a direct connectivity (endpoints know about each other)

The **two level architecture (1b)** the device is unaware of addressing and the communication is handled by gateway. Used for low power devices

Finally the **two level architecture (1c)** implements the TCP/IP protocol stack with a router that manages connections. Usually endpoints (devices) are not resource-constrained.

There are many topologies:

- **Bus:** Every node taps in a common medium. Direct communication between all nodes possible at risk of higher collision probability, there is a **single point of failure**
- **Star:** There is one central (master) node which connects every other (slave) device. Used for small networks (LAN) or Bluetooth. The master is the single point of failure, moreover it acts as bottleneck
- **Ring:** Arranged in a ring. Every node acts as a repeater, again the medium is the single point of failure, but this eliminates collisions and increases throughput. A token integrity mechanism must be implemented
- **Mesh:** Every node directly connects to every other node, it needs $n(n - 1)/2$ links. If this is not satisfied it is a partial mesh

Data

The sensor data is not directly ready to use. We might need sampling, aliasing, quantization, saturation, hysteresis, calibration and error propagation.

Sampling and Aliasing:

Recall the **nyquist thm**: A signal sampled at frequency w can be reconstructed if $B \geq 2w$

If this theorem is not satisfied then we have **aliasing** in the reconstructed signal, that is, high frequency components are lowered in frequency

Quantization and Saturation:

A digital signal can represent each value with a limited number of bits, therefore continuous signals become discrete values. If the value is higher than the maximum allowed by the bits, it gets clamped

Hysteresis, Non-Linearities, Calibration and Error Propagation:

Hysteresis is when a cycle is completed but doesn't return to the original output (0 pressure is 0V, I apply and remove pressure then I have 0 pressure = non zero V. It takes a bit to readjust). Moreover physical phenomena don't necessarily follow a linear relationship in the response.

Calibration is when a sensor is biased and the outputs must account for that.

Some measurements are given by more sensors and also by noise. These errors add up.

The **publish/subscribe** paradigm is used as a continuous data streams as it occupies bandwidth and uses more energy.

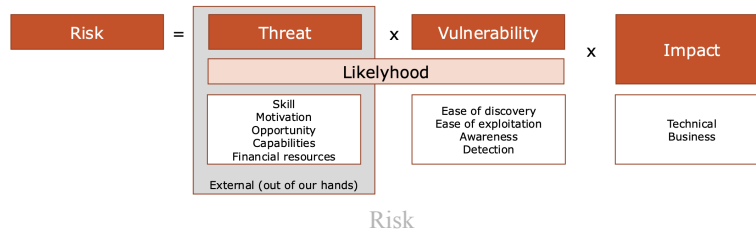
If we know how the model behaves we can **interpolate** data to allow for even less transmissions, but this might lead to underfitting (don't follow correctly) or overfitting (noise)

Also **spatial/temporal correlation** can help with data estimation (traffic at one part of road is roughly same 100m after at the same time), but random nature of events can cause this to be inaccurate (having a lighter doesn't cause cancer but since most smokers have it they tend to develop cancer)

Control Plane

The control plane is the overview of the whole stack.

- The (cyber)security must first identify the risk:



Here are some of the main threats:

Attack/Threat	Purpose
Eavesdropping	Learning confidential info
Traffic Analysis	Learning origin, destination, length but not content
Forgery	Building fake message pretending it was sent by someone else
Masquerade	Claiming to be someone else
Repudiation	Denying to having sent/received a message
Profiling	Gathering info about a single user
Fingerprinting	Identifying user associated to message
Jamming	Causing interference to a communication

Specific standards are adopted for the specific fields.

The lifecycle of an endpoints begins with its **provisioning**: The process of giving the endpoint the configuration data and the ability to present itself. Then it must be **authorized and authenticated**.

During provisioning a secret key is exchanged. A challenge C and SeqN are generated and the server calculates:

- R: expected response
- K: tmp auth key
- Auth: proof that also server has the secret key
This is included in a message $E_k(SeqN)$ that gets sent to the client. If the client computes K, decodes SeqN and if it sends R to the server, the server verifies identity of client.

Two main problems arise:

- Only the server and client must know the key, but this is hard to accomplish: Solution **trust chain** by installing the key via firmware from a **Trusted Third Party (TTP)**

- A close device can impersonate the real device: Solution: Create **Physically Unclonable Functions (PUF)** that depends on random characteristics of fabrication process

The **supervision** set of activities to guarantee that the performance and availability levels required by the IoT applications are maintained.

TODO

QoS and Performance

One of the main performance metric is the **delay**.

The One Way Delay (OWD) and the Two Way Delay (TWD) which also includes system delay and processing.

TODO

Energy Constraints

TODO

2) IoT Communication Technologies

2.1) Wireless

Spectrum

Spectrum is a limited resource and thus it must be used fairly and efficiently. There are three types of spectra:

- **Licensed:** belong to private companies
- **Unlicensed (ISM):** free access, but with regulations to follow
- **Reserved:** for state assigned usage

The limits in ISM bands are:

- Effective Radiated Power (ERP), usually 25-500 mW
- Listen Before Talk (LBT)
- Duty Cycle (DC) ratio over 1h: applied to non LBT devices, usually 0.1-1%

2.1.2) Short Range

Short range technologies are those that have a operational diameter of less than 100m

Radio Frequency IDentification (RFID) and NFC

RFID is a method of remotely storing and retrieving data using **RFID tags (store) and RFID readers (read/write)**. This is done via inductive (magnetic) coupling.

The **distance is 15% of the wavelength, therefore low frequencies are preferred.**

It has many applications: passport, cards, store security.

TYPES

The types are:

- **Semi passive:** uses battery, but only activated when coupling happens
- **Active:** always active with battery (Telepass)
- **Passive:** the coupling powers the tag (vicinity or proximity standards)

PHY LAYER

Modulation:

- Amplitude Shift Keying (**ASK**). 1 has high amplitude, 0 has low amplitude.
- **On-Off Keying (OOK)** is used as a special case of ASK, but it is **data dependent**; if too many 0 are sent, the power received is 0.
- Pulse Interval Encoding (**PIE**) where the length of the pulse identifies 0 or 1, data rate dependent on data

MAC LAYER

With many tags also a MAC is needed. To resolve the mac either **slotted aloha** or **binary tree resolutions** are used. Send message using binary tree, when only one tag responds the correct MAC was received

NFC tags are a RFID reader and tag in one. They **operate exclusively at one frequency (13.56 MHz)**

Feature	RFID	NFC
Network type	Point-to-point	Point-to-point
Communication	Unidirectional	Bidirectional (two-way)
Range	Up to 100 m (depending on frequency)	<0.2 m
Frequency	LF/HF/UHF/Microwave	HF (13.56 MHz)
Bitrate	Varies with frequency	Up to 424 Kbps
Power consumption	Varies with frequency	<15 mA
Continuous sampling	No	Yes

Comparison

Bluetooth And Bluetooth Low Energy (BLE)

Bluetooth is a short distance wireless technology. it has 3 classes:

- **Bluetooth Classic:** Basic Rate (BR) or Enhanced Rate (ER) used for file TX, audio streaming, peripheral devices
- **Bluetooth High Speed:** used for video steaming and tethering (hotspot)
- **Bluetooth Low Energy (BLE):** used for wearables, beacons, smart devices $\Rightarrow$ IoT

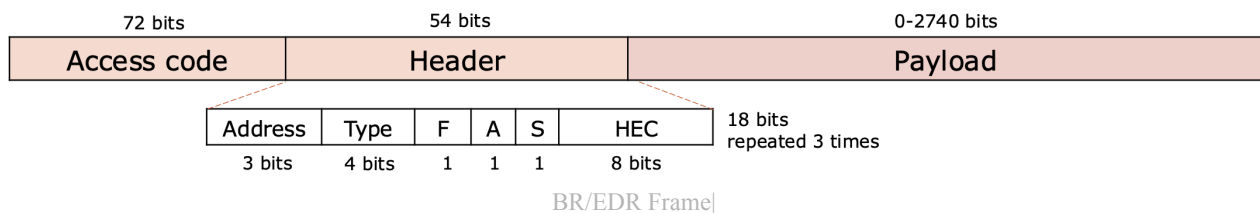
TOPOLOGY

Each BT network is called **piconet** and is a star topology with the master as the central node, the other devices are the slaves.

Up to 255 parked (in sleep) devices, but only 2-8 active devices simultaneously. The channels are accessed via polling.

FRAME

Now we analyze the frame of BT Classic BR/EDR:

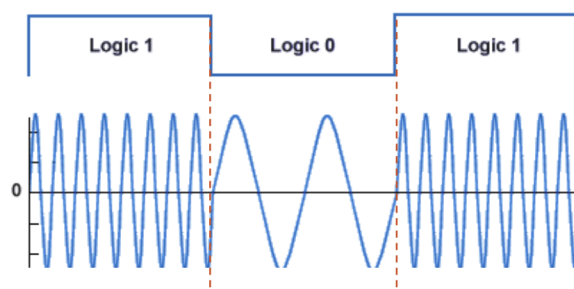


- **Access Code:** Sync bits and piconet ID
- **Header:** 18 bit patten repeated 3 times (coding)
 - **Address:** 3 bits = 7 possible destinations in piconet
 - **Type:** type of message
 - **F:** Flow (1=cannot accept packets)
 - **A:** Ack (S&W, 1 bit enough)
 - **S:** SeqNo (S&W, 1 bit enough)
 - **HEC:** Header Error Correction

RADIO LAYER (=PHY)

Three different modulation schemes:

- **Gaussian Frequency Shift Keying (GFSK) with gaussian filtering:** 1 Mbps. This modulation consists in shifting carrier frequency to represent 0 or 1. Gaussian filtering is used to filter out some of the interference



- **n/4-DQPSK:** 2Mbps
- **8-DQPSK:** 3 Mbps

It operates on the 2.4 GHz ISM band divided into 79 channels of 1 MHz each. Moreover it employs **Frequency-Hopping Spread Spectrum (FHSS)** where it changes the band 1600 times per second, it remains in the band for $625\mu s$ (**dwell time**). See [Baseband Layer](#) to understand how it changes bands.

The TX range depends on the TX power.

BASEBAND LAYER (=MAC)

It uses **Time Division Duplex (TDD)** with slots of 1 dwell time ($=625\mu s$). It consist in alternating the directions for UL and downlink: Master→Slave (even slots), Slave→Master (odd slots).

How do two devices connect?

- **Phase 1; Inquiry:** Master initializes communication link and generates a pseudo random hopping sequence for piconet then goes to sleep
- **Phase 2; Paging:** Master pages a slave. Slave replies with a Device Access Code (DAC), the master responds with the planned frequency hops (might take long). Slave sends another DAC to source

LOGICAL LINK CONTROL AND ADAPTATION PROTOCOL (L2CAP)

L2CAP is used for multiplexing, segmentation, reassembly, QoS and group management. It support two types of communications:

Synchronous Connection Oriented (SCO)	Asynchronous Connection-Less (ACL)
latency more important than integrity.	integrity more important than latency.
symmetric, synchronous	asymmetric, asynchronous
connection oriented	packet oriented
Mac is deterministic (slots reserved at regular time intervals)	Multislot (1, 3, 5), time intervals not regular

We also introduce the

- **Link Manager Protocol (LMP):** This is used to setup and manage Baseband connections
- **Host Controller Interface (HCI):** provides uniform command interface between LMP and Baseband layers
- **Service Discovery Protocol (SDP):** allows devices in a Bluetooth piconet to identify which services are offered by other devices and to learn their characteristics. Operates with request/response paradigm
- **Profiles:** specific tuning settings

BLUETOOTH LOW ENERGY (BLE)

BLE is a more recent extension aimed to low power devices.

- Only uses 40 (2 MHz) channels (vs 79 of 1 MHz)
- Only uses GFSK (2 Mbps) since higher data $\implies$ shorter ToA $\rightarrow$ less energy
- Fewer hops

BLE nodes are of 4 kinds:

- **Broadcaster:** a node which periodically transmits advertisements, but does not allow connections to be established (e.g., iBeacon).

- **Observer:** a node that just listens for advertisements and does not attempt to open connections (e.g., smartphone with an active localization App).
- **Peripheral (og Slave):** a node which transmits advertisements and may accept connection requests, acting as a Slave.
- **Central (og Master):** a node which may open connection towards a peripheral, acting as the Master once the information relative to a peripheral has been received through advertisements.

Feature	BR/EDR	BLE
Power consumption	Higher	Lower
Data rate	Up to 3 Mbps (8-DPSK)	Up to 2 Mbps (GFSK)
Latency	Lower	Higher
Connection Type	Point-to-point	Point-to-point, broadcast, mesh
Use case	Audio, peripherals, ...	IoT, wearables, sensors. ...
Compatibility	Classic Bluetooth devices	Bluetooth 4.0 and newer

Comparison

BLE WORKFLOW

The Stack of BLE consists of

- **Host (Software):** GATP, GATT/ATT, L2CAP, SMP
- **Controller (Hardware/Firmware):** LL+ PHY
- **Host Controller Interface (HCI):** The command/event channel between Host and Controller, HCI commands are sent by the Host and executed inside the Bluetooth controller

The GAP layer differs from client to server:

- **GAP Server:** advertisement
- **GAP Client:** scanning, connecting
It shows the individual data values within a service

The GATT+ATT layer is a collection of related characteristics (like a folder)

Suppose we have server and client scenario:

First SERVER should advertise: broadcasts packets to announce presence and services

The CLIENT scans: sends connection requests to all nearby devices

Once the server is found the CLIENT connects to it: LL starts bidirectional communication

Now at GATT Layer MTUs are exchanged

Now the Services must be discovered via the ATT, and then the services can be explored. Th Device Information Service gives the characteristics of the device.

It works by: Sending ATT request, receiving ATT. Same for attributes

IEEE 802.15.4

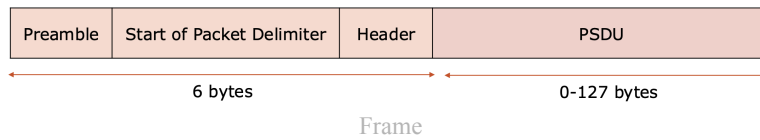
This standard identifies Low-Rate Wireless Personal Area Networks (LR-WPANs). At **PHY+MAC** layers.

it supports large amount of nodes (much more than BLE) of types:

- **Full Function Devices (FFD)** have PHY and MAC capabilities and act as PAN coordinators
- **Reduced Function Device (RFD)** can only talk to FFD and therefore must have star topology

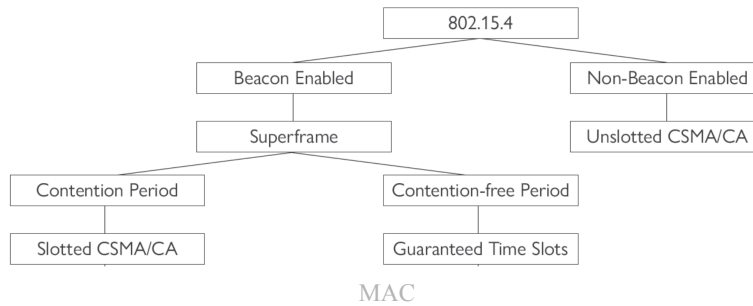
FRAME STRUCTURE

- Preamble: 32 bits for synchronization
- Start of Packet Delimiter: 8 bits
- PHY Header: PSDU length 8 bits
- PSDU: Data field, 0-1016 bits (127 bytes)

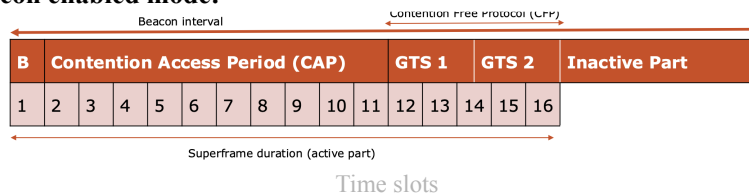


MAC LAYER

At the MAC layer the following scheme is used:



Let's start with the **beacon enabled mode**:



Time is divided into beacon intervals that provide timing for synchronization. The 16 time slots are the **active part** where data exchange happens.

- **Content Access Period (CAP)** uses CSMA/CA
- **Contention Free Protocol (CFP)** is optional and the nodes allocated to the two GTS periods are selected and don't risk collisions

The **inactive part** length is determined by the duty cycle ($\in [0.006, 100]\%$)

In the **non-beacon enabled mode** a device explicitly requests to TX and waits for reply from PAN coordinator. Unslotted CSMA/CA is used and active scanning is required. This **removes beacon listening cost** but active scanning is required.

ZigBee

ZigBee is a standard that builds on top of IEEE 802.15.4 PHY and MAC layers providing **Network and App layer support**. Mainly the creation, joining and leaving of networks, new device configurations, addressing, routing and security

TOPOLOGY

Two main **topologies**:

- **Star**: FDD is coordinator and end devices re FDD or RFD
- **Mesh**: Special FDD used as router (connected to mains) that forwards relays data to standard FFD and RFD

ADDRESSING

Each device has two **addresses**:

- **Extended (IEEE) address**: equivalent to mac, it is assigned at production 64-bit
- **Short address**: equivalent to IP. It is selected by PAN coordinator 16 bits, 65 k nodes $\gg$ BLE

ROUTING

the **routing** is one of the two

- Hierarchical: follows tree structure established at creation of mesh
- Dynamic: Uses Ad-hoc On-demand Distance Vector (AODV) protocol

APP LAYER

APP: manages mapping between 64 and 16 bit addresses, executes fragmentation and reassembly, security

ZDO: special application that defines roles of devices, manages network formation and management and device discovery

AF: Hosting and exposes ZDO to internet

6LowPan

Implements IPv6 over IEEE 802.15.4

Standard IPv6 edge routers can be used to access the internet.

However IPv6 is not a natural fit as:

- Header occupies 1/3 of MTU (40 bytes of 127) (**header compression**)
- Minimum IPv6 frame is 1280 bytes ($\gg$ 127 bytes) (**fragmentation**)
- IPv6 assumes single broadcast, which is not always true.

HEADER COMPRESSION

The header can be compressed by noticing redundant datas:

- Version is always 6 $\rightarrow$ remove
- Payload length can be derived from LL $\rightarrow$ remove

HC1 format uses:

- Src/Dst address: 2 bits, one bit for link local and one for retrievable form LL
- TF: 1 bit if Traffic Class and Flow Label = 0
- NH: 1 bit if next header is known (UDP,TCP,ICMP)
- HC2

This works only in link local scope, outside **IPHC** is used:

- HLIM: Hop limit (1,255 or carried inline)
- CID: Context Identifier, 0 if same LoWAN otherwise 1 and 2 new 4 bit fields
 - DAC/SAC: 1 bit: 0 stateless, 1 stateful
 - DAM/SAM: number of elided address bits

FRAGMENTATION

The IPv6 MTU gets fragmented into IEEE 802.15.4 MTUs and the PSU header identifies the first fragment, then a 16 bit tag is given to all the fragments of same MTU and finally an 8 bit offset is given to each subsequent fragment.

ROUTING

Since IPv6 routing is resource intensive, a new protocol is needed: Routing Protocol for Low Power and Lossy Networks (RPL)

This protocol is created proactively regardless of metric

1. the **DODAG Information Object (DIO)** message is sent from og node to neighbor nodes (DL Path)
2. nodes respond with **DODAG Advertisement Object (DAO)** (UL Path)
 1. storing mode, DAO sent to og node
 2. non storing mode, DAO sent all the way to root
3. og node responds with DAO-ack and each node computes the rank based on objective function

✓ Todo

IPv6

2.1.3) Long Range

Long range spans several kilometers. Usually done by cellular networks but these require high complexity and costs and are not power efficient. A Low Power Wide Area Network (LPWAN) is an alternative.

Parameter	WSN	WLAN	Cellular networks	LPWA
Range	Very short (~1-3 m)	Short (~10-100 m)	Long (~1-5 km)	Very long (~5-20 km)
Energy consumption	Very high (tx. power < 1 mW)	Medium (tx. power ~80 mW)	Low (tx. power ~500 mW)	Very low (tx. power ~ 20 mW)
Data rate	Low	Very high	High	Very low
Cost	Low	Medium	Medium	Very low

Comparison

Sigfox

First LPWAN developed in France via subscription based model for sigfox cloud platform.

Very simple **Ultra Narrow Band (UNB)** to be interference immune and reduce noise power.

PHY LAYER

Each message sent thrice using Differential Binary Phase Shift Keying (DBPSK) where a 1 is encoded as a phase shift, and GFSK

FRAME STRUCTURE

- **Uplink frame:** payload up to 12 Bytes.
- **UL is more common than DL (e.g., for sensors to report data measurements).**

Preamble	Sync.	Device ID	Payload	Auth.	FCS
4 bytes	2 bytes	4 bytes	0÷12 bytes	variable	2 bytes

- **Downlink frame:** payload up to 8 Bytes.

Preamble	Sync.	Flags	FCS	Auth.	Error codes	Payload
8 bytes	13 bits	2 bits	1 byte	2 bytes	variable	0÷8 bytes

Frame

with 1% DC I have 72 Bytes/hour to send (very limited), good for simple apps but NO command-control applications.

MAC LAYER

Unslotted aloha (maximum simplicity).

End device wakes up sends UL, then listens for DL (ACK) and goes to sleep.

LoRaWAN

Long Range Wireless Access Network (LoRaWAN) is a protocol developed in 2009 for IoT long distance communication. LoRa is the physical implementation, while LoRaWAN is the Rest of the protocol stacks.

Let's start with the PHY layer:

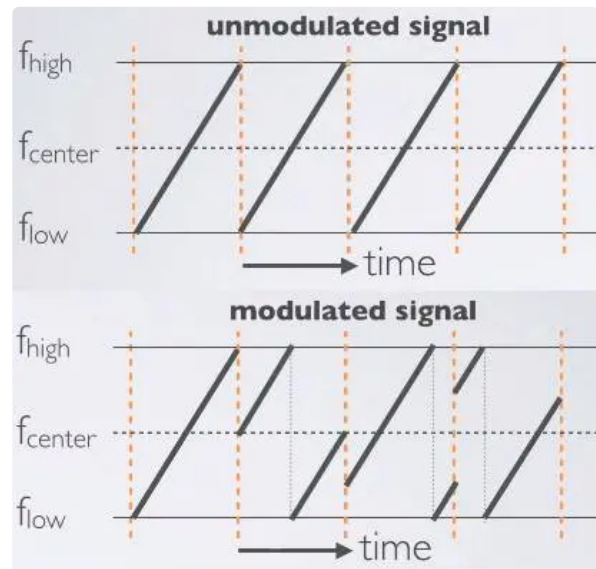
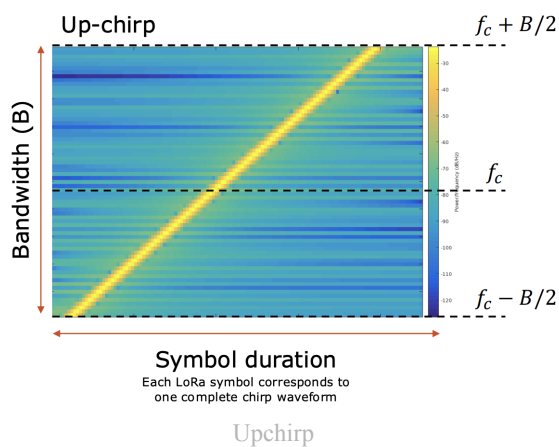
FREQUENCY RANGE

It uses lower frequencies (radio) in order to allow for higher distance communication but lower throughput. **Usually around 868 MHz.**

It uses **24-80 channels of 125 KHz each**. The minimum number of channels is 3 since they are required for the join request.

MODULATION

It uses the **Chirp SpreadSpectrum (CSS)**. Essentially Symbols are encoded by modulating a carrier that changes frequency linearly in time. It has the advantage to spread the power on a wider spectrum in order to resist noise. It is divided in up and down chirps. The slope is always the same, however based on where it starts (and then wraps around) encodes a different symbol. How many symbols are encoded? Look at [SF](#Spreading Factor)



Modulation Example

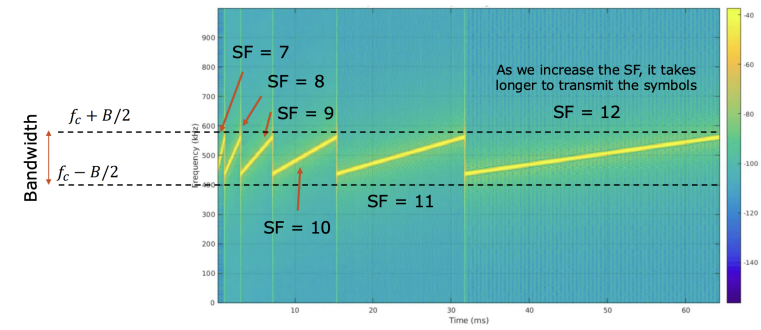
SPREADING FACTOR

The spreading factor **SF** is a LoRa-specific parameter that defines **how many chips are used to encode one symbol**. Therefore each symbol is made of 2^{SF} chips. What this means is that a **chip is the resolution of the waveform**.

LoRa can assume SF values between 7 to 12 and this influences data rate, time on air, battery life and receiver sensitivity

The chip rate is fixed (resolution constant) and equal to the bandwidth, however, clearly, the symbol time changes linearly with the chips.

Symbol Time	Symbol Rate	Bitrate	Energy Consumption
$T_S = \frac{2^{SF}}{B}$	$R_S = \frac{1}{T_S} = \frac{B}{2^{SF}}$	$R = SF \cdot R_S = SF \cdot \frac{B}{2^{SF}}$	$E \propto \frac{SF}{2^{SF}}$
		$R = SF \frac{B}{2^{SF}} \frac{4}{4 + CR}$	More ToA = More energy



Visualization of SF

These are **pseudo orthogonal signals**, this means that multiple signals with different SF can arrive concurrently, since they have different data rates. In practice also on the same SF can be avoid collision if power difference is ≥ 6 dB. Moreover by nature this is a **pure ALOHA**.

LoRa also applies a Forward Error Correction (FEC) by adding a number $CR = \{1, 2, 3, 4\}$ of extra bits every 4 bits. Therefore a symbol $SF - CR$ information bits and the (information) bitrate is decreased (80% up to 50% of R_S).

Higher SF $\iff$ longer ToA $\implies$ We can accumulate energy for longer $\implies$ Better sensitivity and Longer Range. In fact the following table shows how a lower SNR is required for higher SF:

SF (bits/symbol)	SNR_m
7	-7.5 dB
8	-10 dB
9	-12.5 dB
10	-15 dB
11	-17.5 dB
12	-20 dB

SF vs SNR

While this shows a comprehensive list of all useful info, including Data Rates (DR) that go from 0 to 6.

SF	DR	Bandwidth (KHz)	Bit rate (Kbit/s) (CR = 1)	Range (km)	ToA (ms) (10-Byte)	Sensitivity (dBm)	Max. payload at app. (B)
7	DR6	250	10.9	2	28	-120.5	222
	DR5	125	5.5	2	56	-123	222
8	DR4	125	3.125	4	103	-126	222
9	DR3	125	1.758	6	205	-129	115
10	DR2	125	0.977	8	371	-132	51
11	DR1	125	0.537	10	741	-134.5	51
12	DR0	125	0.293	12+	1483	-137	51

Full Table

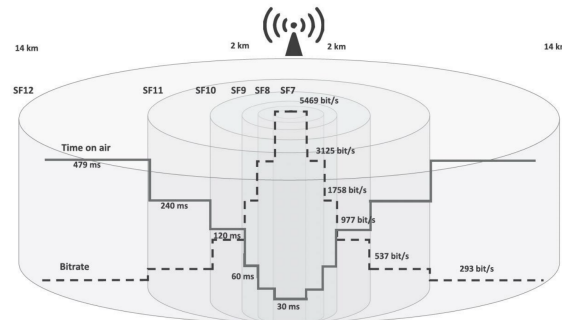
The sensitivity is calculated via

$$S[dBm] = -174 + 10\log(B) + NF + SNR_m$$

And notice that the max. allowed payload size as the application layer decreases as the SF increases in order to reduce the ToA (since it still naturally increases as SF increases).

ADAPTIVE DATA RATE

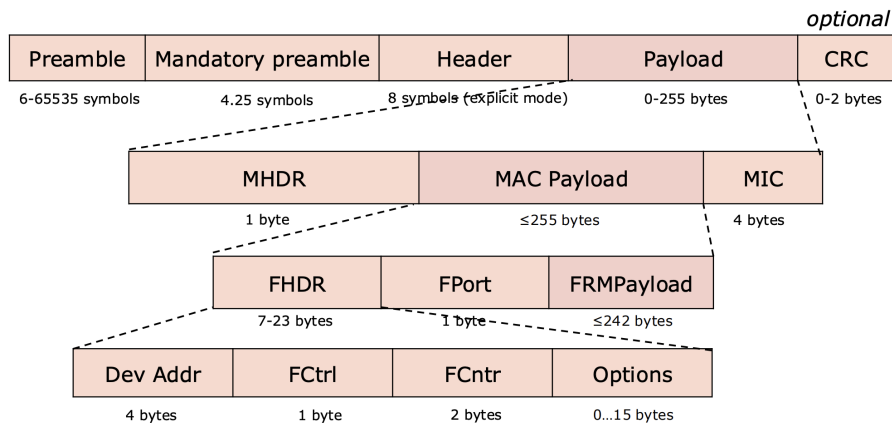
It is important to be able to automatically select the DR based on the conditions of the device. The SF should be high enough to reach the end device, but should avoid energy waste. The rule is: **as the distance of the end device from the gateway increases, the SF increases**



Adaptive Data Rate Visualization

FRAME STRUCTURE

Here is the complete structure of the LoRa Frame



Complete Frame Structure

where:

- **Preamble:** Well known sequence of up-down chirps (usually 8 symbols)
- **Mandatory preamble:** for synchronization. 4.25 symbols means the last chirp is stopped after 1/4
- **Header:** Payload length and other useful info, in implicit mode it is omitted
- **CRC:** Cyclic Redundancy Check
- **MHDR:** MAC Header, specify message type
- **MAC Payload:** This has Min length of 12 +1 bytes as MHDR (1), Dev Addr (4), FCtrl (1), FCnt (2), MIC (4) + Fport(1)
- **MIC:** Message Integrity Code, digital signature
- **FPort:** Frame Port, defines app port number
- **FHDR:** Frame Header:
 - **Dev Addr:**
 - **FCtrl:** Frame Control: control flags, such as whether to use the data rate specified by the gateway for uplink transmission, whether this message acknowledges the reception of the previous message, whether the gateway has more data for the mote.
 - **FCnt:** Frame Counter, for security and replay protection
 - **Options:** Frame options, commands to change data rate, power consumptions etc

The entire ToA is the sum of all bits

$$ToA = (n_{preamble} + 4.25 + n_{payload})T_s$$

where $n_{preamble}$ are the symbols in the preamble and $n_{payload}$ are symbols in Payload+Header+CRC and can be calculated as:

$$n_{payload} = 8 + \max \left\{ \left\lceil \frac{8PL - 4SF + 28 + 16CRC - 20H}{4(SF - 2DE)} \right\rceil (CR + 4), 0 \right\}$$

where:

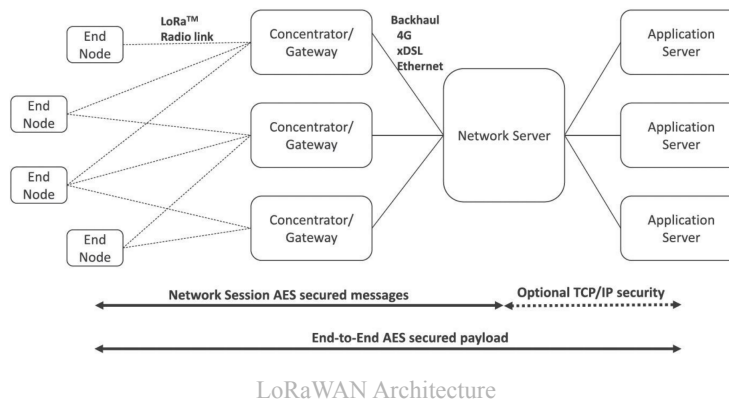
- 8: Min required payload
- 8PL: Mac Payload ≤ 255 B
- 4SF: Compensation factor as LoRa symbols don't carry linear number of information
- 28: internal padding and framing overhead
- 16CRC: If CRC is enabled it is 2 bytes=16 bits
- 4: LoRa interleaves at 4 bits per symbol
- SF: Each Symbol encoded in SF
- 2DE: Low Data Rate Optimization

Moreover further constraints are applied, such as

- **1% Duty Cycle**
- **Fair Access Policy:** 30 seconds uplink time every day per device

TYPES OF END NODES

Now we dig into the LoRaWAN specific Layers



- **End Nodes:** carry sensing and actuating tasks (DevEUI)
- **Gateways:** Stars of start topology (DevAddr)
- **Network Server:** eliminates duplicate messages, routes data, determines the optimal data rate, selects the best Gateway to reach the End Node in DL

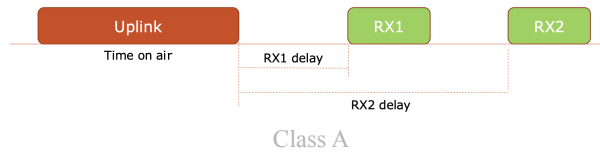
There are 3 types of classes of end nodes:

Class	A	B	C
Communication Type	Initiated by End Node	Bi-Directional	continuous receive
Pro	Very low energy consumption	Lower bounded DL latency	Very low DL latency
Cons	Very high DL latency	Higher Energy consumption	Very high power consumption

CLASS A

Communication always initiated by the End Node

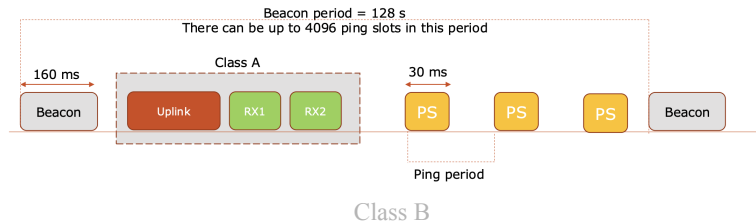
End node sends an UL (uplink) message at an unspecified time (async). It then opens two RX windows for eventual DL. RX2 is opened only if there is no response during RX1.



CLASS B

Communication is bi-directional

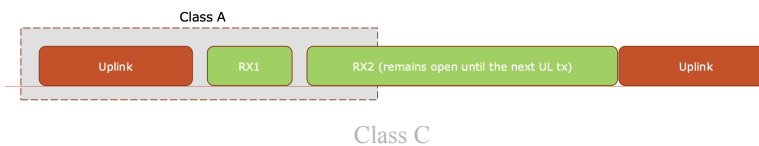
It incorporates class A but adds extra time synchronized beacons sent by the gateway to open ping slots. A beacon is sent every 128s and lasts for 160ms, a ping slot lasts 30ms. The interval between the two beacons (128 s) contains 4096 ping slots.



CLASS C

Continuous receive mode

RX2 remains open until next UL, unless there is activity on RX1



NB-IoT

It stands for Narrow Band - IoT as it uses a small slice of the cellular network. In fact they provide a cost effective way to implement IoT on already existing infrastructure. To conform cellular to IoT requirements two options are viable:

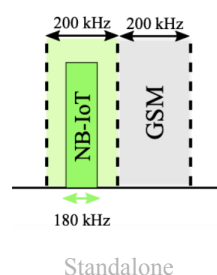
- **Adapting:** use cellular as is but due to higher signal power, less efficient
- **Specifically designed:** NB - Iot

It was standardized by 3GPP (cellular mobile standards) and it is based on 4G-LTE (but not intercompatible, it is a new infrastructure). In fact it uses a set of frequency band in the LTE spectrum and therefore it uses **licensed bands**. More costly but better performance than LoRa (unlicensed). These bands are of 75, 25 or 30 MHz, but the usage of the entire band is resource intensive and might cause more congestion.

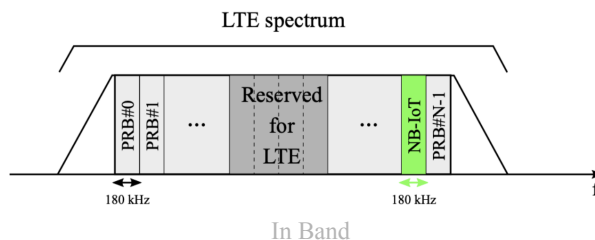
OPERATION MODES

There are 3 main operation modes for NB-IoT:

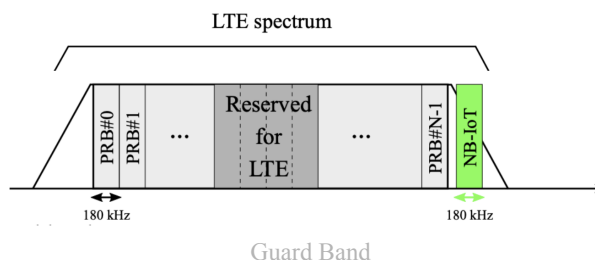
- **Standalone:** NB-IoT occupies the liberated spectrum for GSM (2G) and uses 180 KHz with 10KHz of band guard on each side to occupy the 200 KHz GSM carrier. This is called **refarming** as GSM bands are usually unused and the spectrum becomes dedicated to NB-IoT.



- **In-Band:** Use LTE carrier (1.4-20 MHz wide) and each Physical Resource Block (PRB) has width of 180 KHz and we use one of those for NB-IoT. **Most privileged/cost effective mode** as no refarming is needed, we just use LTE in standard way. But each PRB that is used removes performance of LTE so a limited number of PRBs is available



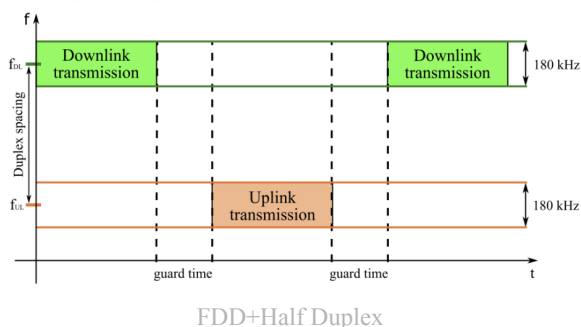
- **Guard Band:** We use the unused guard band of LTE. This might allow for ISI but doesn't impact LTE performance



PHY LAYER

Therefore NB-IoT is similar to LTE:

At **PHY** it uses OFDMA with QPSK or BPSK. 680 bits downlink; 1000 bits uplink with transmit power of 23 dBm and uses FDD mode only with half-duplex. Up/down is on two different carriers and cannot happen at same time

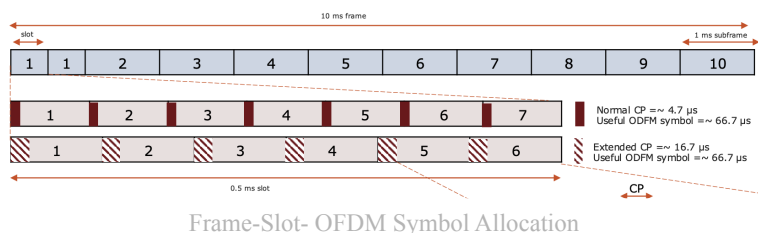


FRAME STRUCTURE

Since OFDM we must distinguish Time and Frequency:

Time:

A frame is 10 ms, divided in 10 subframes of 1ms. These subframes are divided in two of 0.5 ms that contain 6/7 OFDM symbols (66.7 μ s) with CP at start of symbol



Frequency:

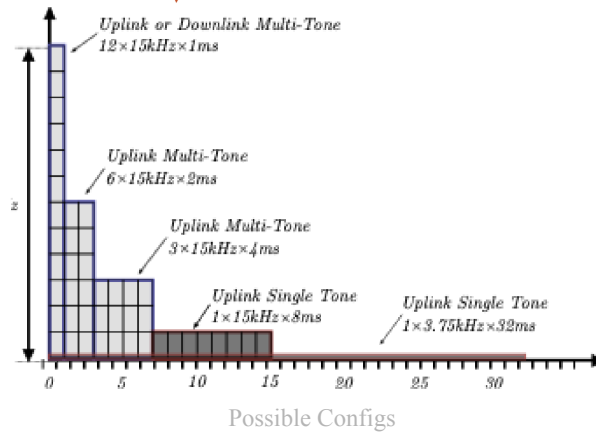
In frequency every Physical resource Block (PRB) is split into 12 subcarriers of 15 kHz. In fact $12 \cdot 15 = 180$ kHz.

Single Tone (Uplink): Every UE (user equipment) uses 1 subcarrier. This is very robust with narrowband-low noise- high SNR and wide coverage.

- 15 kHz subcarrier: 12 subcarriers and slot duration of 0.5ms
- 3.75 kHz subcarrier: 48 subcarriers and slot duration of 2 ms

Multi Tone: UE uses multiple carriers of 15 kHz (increase data rate, less delays/power consumption). And 3, 6 or 12 tones (subcarriers) are assigned to each UE.

So there is 2 degrees of freedom, one in time (slots) and one in frequency (# of subcarriers).



Subcarrier spacing	# tones in RU (subcarriers)	# slots in RU	# symbols in slot	Slot duration	RU duration
3.75 KHz	1	16	7 (normal CP)	2 ms	32 ms
15 KHz	1	16		0.5 ms	8 ms
	3	8			4 ms
	6	4			2 ms
	12	2			1 ms

Configs

the Resource Unit (RU) is a UL resource mapping unit that is a combination of slots in time and frequency.

MAC LAYER

Power Saving Mode (PSM):

Disables part of protocol and goes in idle mode (μA range) but remain connected.

- Sleep Mode: Can still receive DL paging options (PSM Timer)
- Deep Sleep Mode: cannot receive DL (radio shut down) but network is aware of device (TAU Timer > PSM)

Extended Idle Mode DRX (eDRX):

Sleep mode with periodic wake up to check for paging. It is used to receive more frequently data than in PSM.

These two modes can be combined.

Repetition:

Based on signal strength a DL/UL message is sent 2048/128 times.

Each doubling of the repetition increases sensitivity by 3 dB and can therefore extend coverage but adds latency. The number of repetitions is dynamically adjusted by an Enhanced Coverage Level (ECL). It is based on the Maximum Coupling Loss (MCL) of 164 dB.

2.2) Wired

Ethernet is based on the IEEE 802.3 standard with Length field becoming Type in header.

It has the following features:

- Connection less: sends when ready
- No flow control: if too many packets they are silently dropped
- No ACK (no ARQ): if CRC fails packets are lost
- CSMA/CD

FRAME FORMAT

18 bytes header + 46-1500 payload bytes

- The max frame length is to avoid monopolization of bus
- Minimum is to allow CSMA/CD to work since $T_{tx} > 2T_p$

Industrial Ethernet

Ethernet is not deterministic and might not support real-time applications Industrial ethernet fixes this and also provides more robust communication and cables

CLASSES

Class 1: Ordinary Ethernet + TCP/IP Stack

- Full compatibility with standard Ethernet
- "Soft-real time" (latency ≥ 100 ms)
- Unpredictable latency

Class 2: Ordinary Ethernet + Custom Apps

- Runs special designed protocol stack (latency ≥ 10 ms)
- Some protocol modification still needed for real-time
- cannot route through IP network

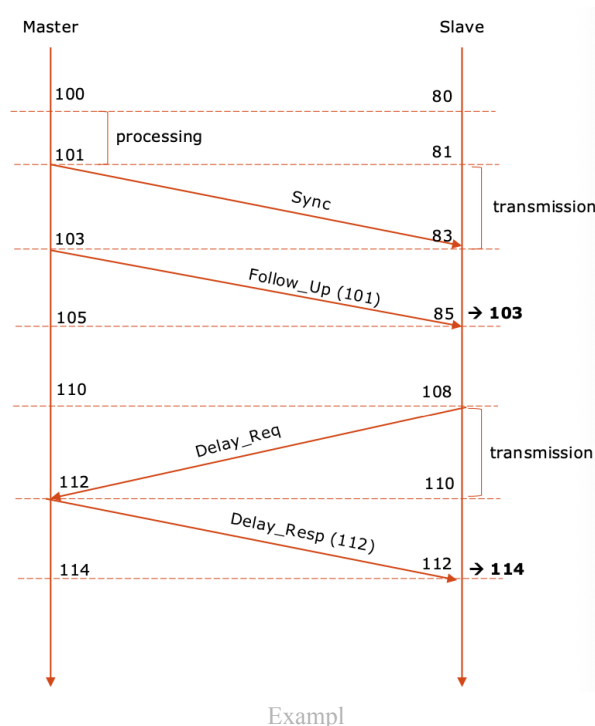
Class 3: Modified Ethernet + Custom Apps

- Support hard real time requirements (latency ≥ 1 ms)
- Modifies og ethernet
- Needs custom equipment
- cannot route through IP network

IEEE 1588 PRECISION TIME PROTOCOL (PTP)

This protocol is used to send timing signals to let the tx and rx to be synchronized

- Sync signal is sent by Master (M) to Slave (S) with an offset (O)
- A follow up signal with the tx time of the sync (T_{sync}) is received and the new time is calculated as: $T_{sync} - rx \text{ time} = \text{offset}$. and thus the clock adds the new offset and the final offset is only the tx delay
- Delay req is sent and a response with the arrival time is received. This lets S calculate the delay and then adds half of it to the clock

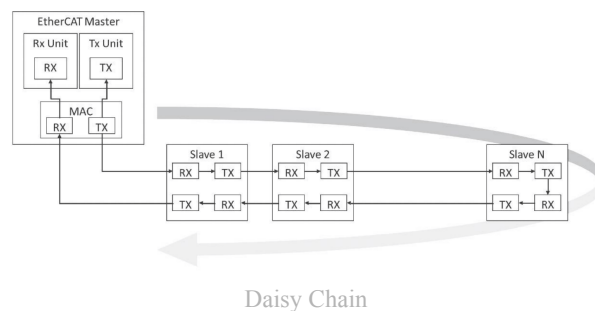


Single pair ethernet uses one pair of twisted wires instead of 4 pairs, this allows for looser data rate but higher range and lower costs and space

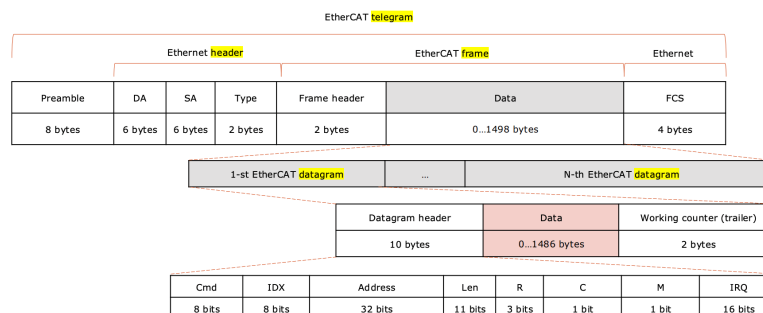
EtherCAT

Special implementation of Ethernet for industrial applications class 3

It avoids the use of switches by keeping a daisy chain master slave ring but requires ad-hoc hardware



The slaves only read the necessary bytes meant for them, one single message can be intended for multiple slaves so slaves insert/extract data on the fly.



Frame

Each slave has a structured internal memory called EtherCAT Slave Controller (ESC) and a Fieldbus Memory Managment Unit (FMMU)

1. EtherCAT frame arrives at slave
2. ESC checks FFMU
3. ESC performs on-the-fly processing using FMMU
4. ESC forwards frame

The working counter gets increased when ESC modifies content, while the Address is the daisy chain number of the slave.

ADDRESSING

Position-based addressing

The addressing is done at startup, then the address is decreased by 1 each time and the slave that gets address 0 is selected

Configurable addressing

stored in ESC as 16-bit configurable address, used for non cyclic communication

Logical addressing

specifies where in the frame the read/write is done

Broadcast

all slaves receive the frame

3) IoT Applications

This part focuses on application/transport layers.

UDP and TCP alone do not suffice in complex environments, we use a **middleware** to complement transport protocols

A middleware is needed since no single solution satisfies all possible IoT requirements.

3.1) WebSocket

the World Wide Web (WWW) is a distributed **client-server** service, that is a client using a browser accesses a service using a server. A Uniform Resource Locator (URL) is used to determine what server to fetch the files from. HTTP is not compatible with IoT as it is stateless (cannot accomodate event-based updates), not asynchronous, and very big headers

WebSocket is a combination of 2 elements

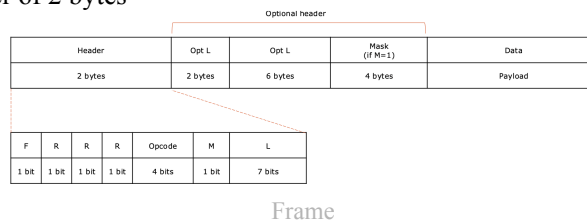
- Open bidirectional TCP connection for data exchange using HTTP
- Minimum set of messages that allow to maintain and transfer the connection

HTTP GET request to open connection and if WebSocket is not supported an error message is returned.

Websocket are symmetrical so both client-server can send messages, however it is still client-server approach.

Frame Structure

WebSocket has minimal header of 2 bytes

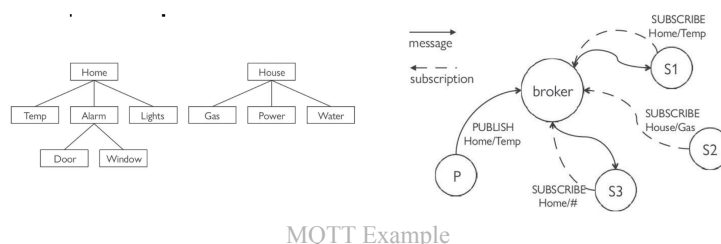


- F: if 1 indicates last frame or full segment
- R: reserved
- Opcode: type of message
- M: 1 if a mask is used in payload
- L: length of payload
 - $L < 126$
- Mask: used for client-server comm, marks packets with key for security

3.2) Message Queuing Telemetry Transport (MQTT)

MQTT is a IoT specific app protocol. It is layered on top of TCP or TLS and implements a publish-subscribe paradigm.

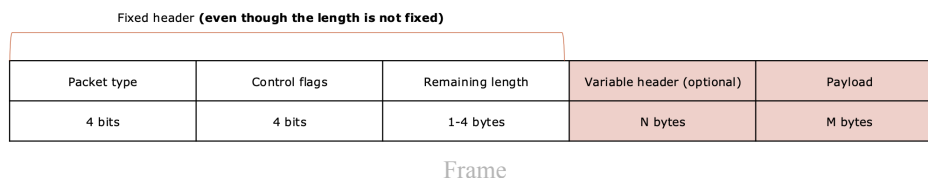
The base idea is the following: There is a central publisher that distributes messages to a broker. The broker forwards them to the subscribers. The subscribers never establish connections between each other. The subscriptions are made to specific topics.



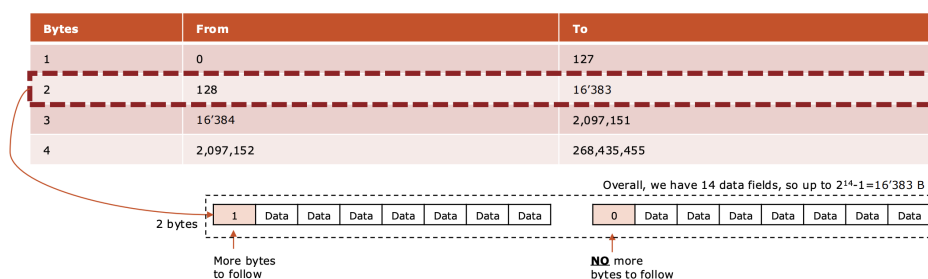
This protocol is lightweight, can be implemented on many platforms, has retain and will mechanism. However the centralized broker might create scalability issues.

Frame Structure

MQTT messages are characterized by a minimal header for efficiency.



- **Variable Header / Payload:** depends on message
- **Remaining Length:** represents sum of lengths of variable header and payload fields. Encoded using **Variable Byte Integer** which deploys a number of bytes based on represented int
- **Variable Byte Integer:** each byte encodes 7 bits of data and uses most significant bit as **continuation flag**. (1=more bytes, 0=no more bytes). Max is $2^{42} - 1$ bits $\approx$ 268 MB



Example

First we start with some basic definitions:

RETAINED MESSAGE

Since it is a client-server paradigm, subscribers cannot directly fetch data from publishers, but the latest data is obtainable through Retained Messages.

A message with the retained data field tells the broker to store the last message, on reconnection the subscribers will get that message.

For example, a data that rarely changes can be sent every T even if the data doesn't change. However, it is possible to send the data only when it changes under the form of a retained message in order to save battery of the publisher.

WILL MESSAGE

If a will message is enabled it will be published when the connection terminates ungracefully. It contains:

- Will Topic: where the broker will publish the message
- Will Message: the message payload

The will message can be sent as a retained message in order to let new subscribers know that the message was sent.

KEEP ALIVE

Since MQTT is based on TCP, we can have "half connection" problems, that is, connection disabled on one side but enabled on another; the data might be correctly sent but never reach the final destination.

The Keep Alive allows the MQTT server to determine if a half-connection problem arises and to then close the connection. Keep Alive is typically used in conjunction with Will Message: if the server does not receive any packets within the Keep Alive timer, it will send a Will Message.

Clearly the client must have $T < \text{Keep Alive}$.

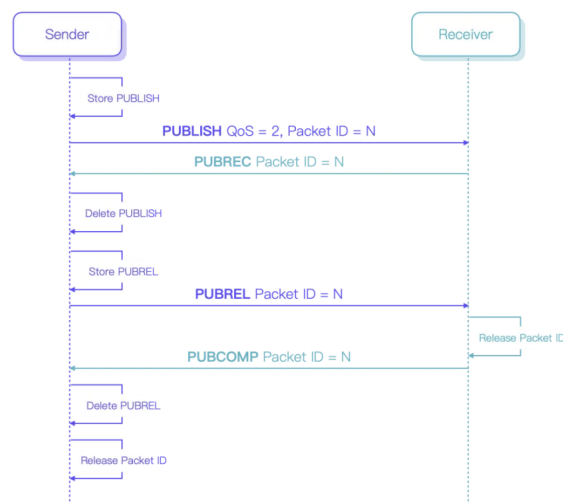
QoS LEVELS

QoS 0	QoS 1	QoS 2
At most once	At least once	Exactly once
The sender does not wait for acknowledgement or store and retransmit the message, so the receiver does not need to worry about receiving duplicate messages.	When the sender receives a PUBACK packet from the receiver, it considers the message delivered successfully. Until then, PUBLISH packet is stored for potential retransmission. Packet ID is used to match PUBLISH with PUBACK	The sender stores and sends a PUBLISH packet with QoS 2 and then waits for a PUBREC response packet from the receiver. This process is similar to QoS 2, with the exception that the response packet is PUBREC instead of PUBACK. The sender deletes its copy The sender sends a PUBREL packet to release the Packet ID. It is stored for potential retx. The receiver responds with a PUBCOMP packet

Remark.

The effective QoS is

$$\text{Effective QoS} = \min \{\text{Publisher QoS}, \text{Subscriber QoS}\}$$



Example of QoS 2

Here are some examples from the terminal on how QoS is seen from a broker perspective

QoS=0

```
Received PUBLISH from pub1 (d0, q0, r0, m1, 'topic/test', ... (4 bytes))
Sending PUBLISH to sub1 (d0, q0, r0, m5, 'topic/test', ... (4 bytes))
```

QoS=1

```
Received PUBLISH from pub1 (d0, q2, r0, m1, 'topic/test', ... (4 bytes))
Sending PUBLISH to sub1 (d0, q2, r0, m5, 'topic/test', ... (4 bytes))
```

```
Sending PUBACK to pub1(m1, rc0)
Received PUBACK from sub1(Mid: 5, RC:0)
```

QoS=2

```
Received PUBLISH from pub1 (d0, q2, r0, m1, 'topic/test', ... (4 bytes))
Sending PUBREC to pub1 (m1, rc0)
Received PUBREL from pub1 (m1)
Sending PUBCOMP to pub1 (m1)
```

```
Sending PUBLISH to sub1 (d0, q2, r0, m5, 'topic/test', ... (4 bytes))
Received PUBREC from sub1 (Mid: 5, RC:0)
Sending PUBREL to sub1 (Mid: 5)
Received PUBCOMP from sub1 (Mid: 5)
```

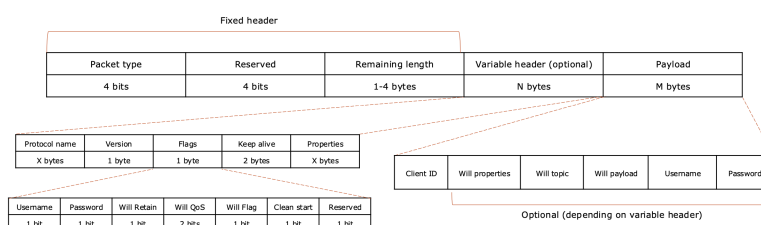
In synthesis, we can identify it based on the messages sent:

QoS Level	Handshake Packets in Log	Reliability	Speed
QoS 0	None (Just PUBLISH)	Lowest	Fastest
QoS 1	PUBACK	Medium (Guaranteed, maybe duplicates)	Medium
QoS 2	PUBREC, PUBREL, PUBCOMP	Highest (Guaranteed, unique)	Slowest

The **packet type** has many options, we just focus on: CONNECT, CONNACK, SUBSCRIBE, PUBLISH.

CONNECT

It deals with connections establishment, control and removal. CONNECT (re)opens a session iwth the broker and carries the ID of the client.



CONNECT Frame

- **Username:** indicates whether payload contains username
- **Password:** indicates whether payload contains password
- **Will:**

- **Retain:** indicates if will is retained
- **QoS:** indicates QoS of message
- **Flag:** indicates if payload contains flag
- **Clean Start:** indicates if session is new or continuation
- **Reserved:**
- **Keep Alive:** indicates time interval between two control packets sent

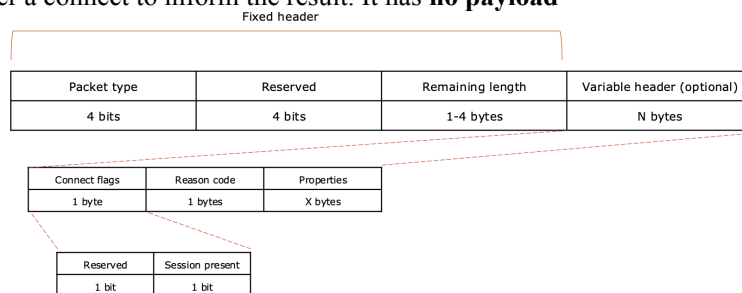
In this example a new connection was established with an username and password. It has a Will that is not retained and keep alive of 600s

CONNECT	
FLAGS	User Name Flag: 1
	Password Flag: 1
	Will Retain: 0
	Will QoS: 2
	Will Flag: 1
Clean Start: 1	
Keep Alive: 600	
ClientID: "S1"	
Will Topic: "Home/Temp"	
Will Payload: "S1 can't track Temp anymore"	
User Name: "users1"	
Password: "rmvt22"	

CONNECT Example

CONNACK

This is the response after a connect to inform the result. It has **no payload**

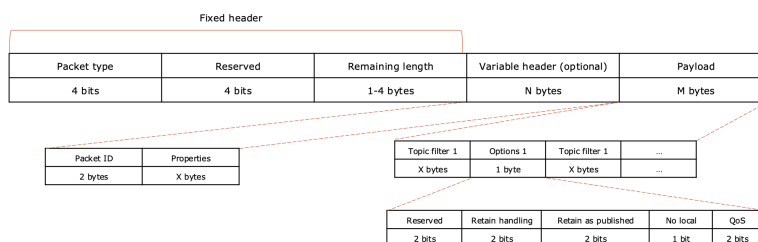


CONNACK Frame

- **Reason code:** indicates result of connection (both for good connections and failed ones)

SUBSCRIBE

It is used to initiate a subscription to a topic with a specified QoS level.



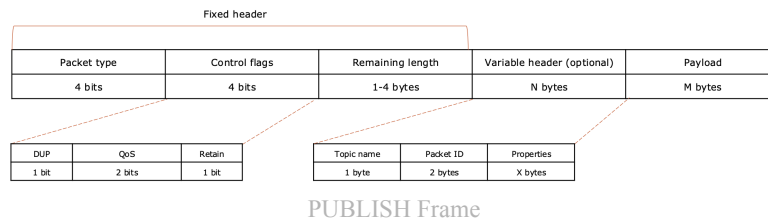
SUBSCRIBE Frame

- **Packet ID:** uniquely identify subscription request. PUBLISH, SUBSCRIBE, UNSUBSCRIBE use a set of identifiers

- **Retain Handling:** if the retained message should be received when the subscription happens
- **Retain as Published:** if the retain flag should be sent along the retained message
- **No Local:** Used to indicate whether the server can forward the application message to the publisher of the message
- **QoS:** max QoS level that the server can use

PUBLISH

This is used to publish the messages: either by client to server or server to publishers. It is followed by PUBACK, PUBREC, PUBREL and PUBCOMP.



- **DUP:** =1 if it is a RETX. The amount of ones received is indicative of the link quality
- **QoS:** Has priority
- **Retain:**
- **Topic Name:** what topic it should be published to
- **Payload:** The content of the message

3.3) Constrained Application Protocol (CoAP)

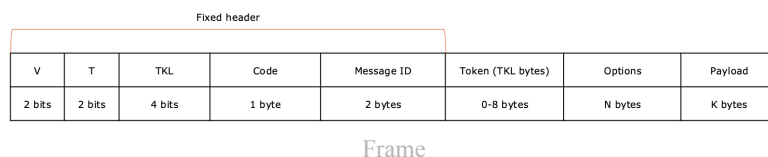
Similar to HTTP as it is a client-server paradigm with request and response.

The communication is either p2p or through a gw. A CoAP server is very resource limited so it can be simply implemented and can also act as a client.

Frame Structure

Since it is built on top of UDP (not TCP like MQTT) it is a bit more complex but still less complex than HTTP

Header of 4 Bytes



- **V:** Version
- **T:** Type; Request (confirmable, non confirmable) or Response (ack, reset)
- **TKL:** Token Length
- **Token:** used to match response with request
- **Message ID:** detects dup or retx
- **Code:** ID of request or response

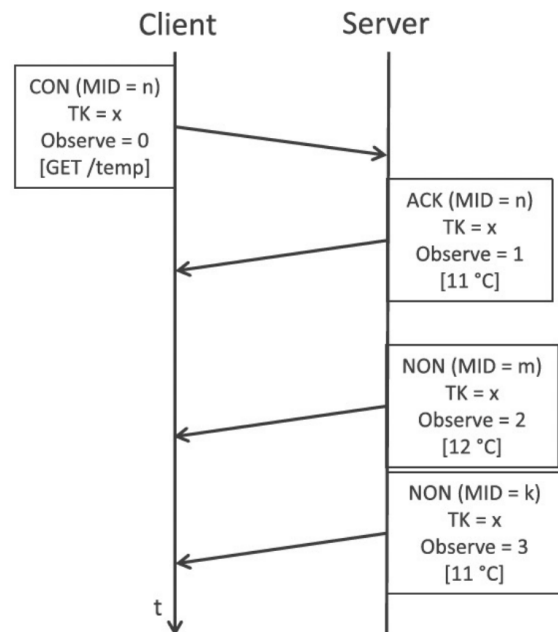
Message IDs: are used to detect duplicate messages and match confirmable messages to acknowledgment messages

Tokens: not always match Message ID since:

- **Separate Response:** token and ack have different message ID, requires one additional ack from client
- **Piggyback:** response sent with ack, so same message ID

Resource Observation

Asynchronously get updates on states of resources.
Client sends GET message with Observer = 0.
Client will receive a series of responses with same Token all carrying info about stato of measure.
This ends after a Reset or a Get with Observer = 1.



Example

Final Comparison

Feature	MQTT	CoAP
Purpose	Messaging and communication in IoT	Designed for resource-constrained devices in IoT
Transport Protocol	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Communication Style	Publish/Subscribe model	Request/Response (Client-Server-like) model
Header Size	2 bytes fixed header	4 bytes fixed header
Payload Format	Supports binary and text payloads	Supports binary and text payloads
QoS (Quality of Service)	Levels 0, 1, and 2 for message delivery	Reliability through "confirmable" and "non-confirmable" messages
Message Types	Publish, Subscribe, Connect, Disconnect, etc.	GET, POST, PUT, DELETE, etc.
Resource Discovery	Not inherent, requires additional mechanisms	Built-in resource discovery through CoRE Link Format

Comparison

4) IoT Cloud

4.1) Data Processing In the Cloud

IoT data can be processed to optimize operations, predict failures, etc...

However the data might contain low quality samples and might be very much, moreover data privacy concerns might be a problem

Data can be processed in the cloud or in the edge

Feature	Cloud computing	Edge computing
Definition	Data is sent to centralized data centers for processing and storage.	Data is processed locally near the source of generation (devices/sensors).
Latency	Higher latency due to transmission to remote servers.	Low latency because processing happens close to the device.
Bandwidth consumption	High bandwidth usage; large amounts of raw data transmitted to cloud.	Lower bandwidth usage; only processed or relevant data sent to cloud.
Processing capacity	Virtually unlimited.	Limited by local device capabilities; may require specialized hardware.
Data storage	Virtually unlimited storage capacity in cloud data centers.	Limited local storage; often temporary or for immediate use.
Cost	Pay-per-use model; can be expensive for high-volume data transfers.	Lower ongoing bandwidth costs; initial cost for edge devices.
Reliability	Dependent on internet connectivity; downtime can impact access.	Can operate offline; less affected by network disruptions.
Security and privacy	Easier to implement some security measures but higher exposure to attacks.	Data remains local (improving privacy), but local security is device-dependent.
Scalability	Easily scalable with cloud infrastructure.	Scalability is limited by the number of edge devices deployed.

Cloud vs Edge computing

Essentially, Cloud can compute more data with better results but comes with more latency and less security (privacy and reliability).

3 types of cloud computing classes:

- Public: all resources given by a commercial cloud that everyone can access through subscription
- Private: all resources managed by enterprise, more control and privacy, less resources
- Hybrid: outsource the most expensive parts

Control loops are employed to operate the actuators based on the data

Machine learning algorithms are employed to analyze data:

- **Supervised learning:** Labeled dataset, appropriately fit model and validate on dataset
 - **Classification/Regression:** simple and can also be run on edge, but cannot describe complex phenomena
 - **Neural Networks:** more complex but can represent non linear models and extracts relevant features
- **Reinforcement learning:** uses a reward function to train and determine accuracy of output. Performs well in heterogeneous environments
- **Unsupervised learning, Clustering:** low weight and identifies hidden patterns

4.2) Cloud Computing Models

Here IaaS, PaaS and SaaS are compared:

	Software as a Service (SaaS)	Function as a Service (FaaS)
	Platform as a Service (PaaS)	Applications: CRM, email, office productivity, games Function instantiation, activation, scaling
Infrastructure as a Service (IaaS)	Language runtime, libraries, database, web servers, development tools	
	Virtual machines	
	Servers, storage, network	

Comp

On-site	IaaS	PaaS	SaaS
Applications	Applications	Applications	Applications
Data	Data	Data	Data
Runtime	Runtime	Runtime	Runtime
Middleware	Middleware	Middleware	Middleware
O/S	O/S	O/S	O/S
Virtualization	Virtualization	Virtualization	Virtualization
Servers	Servers	Servers	Servers
Storage	Storage	Storage	Storage
Networking	Networking	Networking	Networking

Managed By

Feature	IaaS	PaaS
Control	Full control over infrastructure	Limited control, focus on application logic
Management	User manages OS, middleware, runtime, and apps	Provider manages OS, middleware, and runtime
Flexibility	Highly customizable	Pre-configured, less flexible
Skill Requirement	Requires expertise in managing infrastructure	Suitable for developers, minimal infrastructure skills
Use Cases	Data storage, legacy application hosting	Rapid app development, microservices

IaaS vs PaaS

Feature	SaaS	PaaS
Purpose	Provides fully operational software	Provides a platform for developing applications
Target Users	End-users	Developers and IT professionals
Customization	Minimal customization, mostly configuration	High customization for app development
Management	Provider manages everything	Users manage apps and data, while the provider manages the platform
Skill Requirement	None (general use)	Development and deployment skills needed

SaaS vs PaaS

4.3) Cloud Components

The cloud basically performs in these blocks:

- Edge Interface (Cloud Edge Gateway): data injection
- Stream Processing:
- Data Storage:
- Management and Security:

Cloud Edge Gateway

Boundary between edge and cloud that receives big amount of data

Incoming data arrives from IoT devices/gateways and is routed accordingly either to the cloud computing or outgoing to the edge

The outgoing data is config commands or actuation commands

Digital Twin

Cloud might employ a digital twin of IoT devices to test without using bandwidth power and employing physical damage.

Stream Processing

Data is processed sequentially in real time and flows directly to consumer

Storage

Data storage can follow 2 strategies:

- Short term storage: store data for predefined time (hours or days) and allow for fast access to it

- Long term storage: High-precision data can be aggregated and down sampled for trend analysis and archival purpose.

Data is much, arrives fast and from many different types of sensors that can be very noisy or have many errors

Database and API

Data is stored in a distributed database, which can be accessed by the API calls

4.4) Security

The security goals are: **Confidentiality, Integrity, Availability**

There are many vulnerabilities:

- **Physical attacks:** physically altering or damaging measures
- **Software attacks:** viruses, phishing
- **Network attacks:** standard hacking techniques (sniffing, eavesdropping, forgery, etc)

Types of attacks

The two main categories are passive and active attacks

Passive attacks happen without altering data or system behaviour

Active attacks alter data or service operation

PASSIVE

- Eavesdropping: intercepting data in transit to steal sensitive information
 - Weak to: encryption cryptography, distortion, watermarking, secure connections
- Traffic analysis
- Node destruction
- Node malfunctioning
- Node interruption

ACTIVE

- Denial Of Service (DoS): continuously send signals to overwhelm devices, **IoT specifically this causes more battery depletion**
 - Collision: target specific device so to create collision on its packets and cause retx (high battery usage) and harder to notice
 - Jamming: spam messages
 - Exhaustion: prevent entering of low power mode (LoRaWan Class A force to open UL windows)
- Hello Flooding: IoT nodes don't have strong auth, so a strong malicious hello message is seen as legitimate and nodes will connect to it
- Sinkhole: advertise as exceptional router for traffic so all data passes through it
- Wormhole: similar to sinkhole, just by creating a good link, not good router
- Grayhole: drop certain packets
- SYN Flooding: Initiate large number of TCP connection attempts

APPLICATION

CoAP vulnerabilities:

Use same messageID for fake reply, flood with requests or observe subscriptions

MQTT vulnerabilities

paths can be predicatble, create fake will messages

4.4.2) Edge Security